

TEAM REFERENCE DE LOS SURFISTAS BOLIVIANOS

Universidad Mayor de San Simón

Contents

1 Lorem ipsum	2	7 Graphs	8
1.1 Código de ejemplo	2	7.1 DFS	8
2 Lorem ipsum	3	7.2 BFS	8
2.1 Código de ejemplo	3	7.3 Dijkstra	8
3 Data Structures	4	7.4 Floyd Warshall	9
3.1 Segment Tree	4	7.5 Bellman Ford	9
4 Lorem ipsum	5	7.6 Topological Sort	9
4.1 Código de ejemplo	5	7.7 DSU	10
5 Lorem ipsum	6	7.8 SCC	10
5.1 Código de ejemplo	6	7.9 LCA	10
6 Lorem ipsum	7	8 Lorem ipsum	12
6.1 Código de ejemplo	7	8.1 Código de ejemplo	12
		9 Strings	13
		9.1 KMP	13
		9.2 Aho-Corasick	13
		10 Lorem ipsum	16
		10.1 Código de ejemplo	16

1 Lorem ipsum

1.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

2 Lorem ipsum

2.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n\log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

3 Data Structures

3.1 Segment Tree

Descripción: Estructura de datos que permite responder a consultas realizadas sobre un arreglo de elementos. Además de poder realizar cálculos dado un intervalo, esta estructura soporta la modificación de los valores en el arreglo.

Complejidad Temporal: $O(\log(n))$

Usos:

- Suma de elementos en un intervalo
- Hallar el valor máximo/mínimo en un intervalo
- Etc.

```
// Cantidad de elementos en el arreglo original.
int n;

/*
  Vector que representa el arbol binario de forma implícita
  (Tener en cuenta que esta estructura debe manipularse con
  indexación en 1).
*/

vector<int> t(4 * n, 0);

/*
  Al momento de construir un SegmentTree hay que tomar en
  cuenta dos cosas: el valor almacenado en cada nodo y la
  operación encargada de fusionar los valores de los nodos.
*/

void build(vector<int> &a, int v, int tl, int tr){
    if(tl == tr){
        t[v] = a[tl];
    }else{
        int tm = (tl + tr) / 2;
        build(a, 2 * v, tl, tm);
        build(a, 2 * v + 1, tm + 1, tr);

        t[v] = t[2 * v] + t[2 * v + 1];
    }
}

int sum(int v, int tl, int tr, int l, int r) {
```

```
int ans = 0;
if(l > r){
    ans = 0;
}else if(l == tl && r == tr){
    ans = t[v];
}else{
    int tm = (tl + tr) / 2;
    ans = sum(v * 2, tl, tm, l, min(r, tm));
    ans += sum(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r);
}
return ans;
}

void update(int v, int tl, int tr, int pos, int new_val){
    if(tl == tr){
        t[v] = new_val;
    }else{
        int tm = (tl + tr) / 2;

        if(pos <= tm){
            update(2 * v, tl, tm, pos, new_val);
        }else{
            update(2 * v + 1, tm + 1, tr, pos, new_val);
        }

        t[v] = t[2 * v] + t[2 * v + 1];
    }
}
```

4 Lorem ipsum

4.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n\log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

5 Lorem ipsum

5.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

6 Lorem ipsum

6.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

7 Graphs

7.1 DFS

Descripción: Algoritmo de búsqueda que va lo más profundo posible en un grafo con n nodos y m aristas.

Complejidad Temporal: $O(n + m)$

Usos:

- Encontrar para un nodo u algún camino que le permita llegar a cualquier otro nodo del grafo (si es posible)
- Ver si un nodo en un árbol es ancestro de otro haciendo uso de arreglos de tiempos de entrada y salida
- Topological sort usando un arreglo para tiempos de salida
- Encontrar componentes fuertemente conexas

```
vector<vector<ll>> g;
vector<bool> visi;

void dfs(ll v){
    visi[v] = true;

    for(ll u: g[v]){
        if(!visi[u]){
            dfs(u);
        }
    }
}
```

7.2 BFS

Descripción: Algoritmo para recorrer grafos que visita los nodos en amplitud, análogo a la forma en la que se expande un incendio: si un nodo está “ardiendo”, sus vecinos arden después de manera simultanea.

Complejidad Temporal: $O(n + m)$

Usos:

- Encontrar el camino más corto en un grafo sin pesos

- Encontrar todas las componentes conexas en un grafo
- Encontrar el ciclo más corto conteniendo a un nodo origen en un grafo dirigido sin pesos
- Verificar si una arista (u, v) pertenece a uno de los caminos más cortos entre a y b con $d_a[u] + 1 + d_b[v] = d_a[b]$
- Verificar si un nodo v pertenece a uno de los caminos más cortos entre a y b con $d_a[v] + d_b[v] = d_a[b]$

```
vector<vector<ll>> g;

void bfs(ll u){
    ll n = g.size();

    queue<ll> q;
    vector<bool> visi(n);

    q.push(s);
    visi[u] = true;
    while (!q.empty()) {
        ll v = q.front();
        q.pop();
        for (ll u : g[v]) {
            if (!visi[u]) {
                visi[u] = true;
                q.push(u);
            }
        }
    }
}
```

7.3 Dijkstra

Descripción: Algoritmo que permite encontrar las distancias más cortas de un nodo de inicio a todos los demás nodos en un grafo pesado. Los pesos del grafo deben ser todos positivos.

Complejidad Temporal: $O(n^2 + m)$

Usos: Encontrar la distancia más corta de un nodo dado a todos los demás en un grafo.


```

11 INF = 1e18 // Asegurarse que el valor de infinito
           // supere a la distancia más grande posible en el problema
vector<ll> dijkstra(vector<vector<pair<ll, ll>>> &g, ll source){
    ll n = g.size();
    vector<ll> dis(n, 1e9);
    vector<bool> vis(n);
    priority_queue<pair<ll, ll>> que;
    que.push({0, source});
    dis[source] = 0;
    while(!que.empty()){
        ll node = que.top().s;
        que.pop();
        if(vis[node]) continue;
        vis[node] = 1;
        for(ii ed : g[node])
            if(dis[ed.f] > dis[node] + ed.s){
                dis[ed.f] = dis[node] + ed.s;
                que.push({-dis[ed.f], ed.f});
            }
    }
    return dis;
}

```

7.4 Floyd Warshall

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>
```

```
int main() {
```

```

std::cout << "Hola_Mundo" << std::endl;
return 0;
}

```

7.5 Bellman Ford

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>
```

```

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}

```

7.6 Topological Sort

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

7.7 DSU

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

7.8 SCC

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

7.9 LCA

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi

ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

8 Lorem ipsum

8.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n \log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```

9 Strings

9.1 KMP

Descripción: Nos permite buscar todas las ocurrencias de un patrón de un string.

Complejidad Temporal: $O(n)$

Usos:

- Buscar un substring o patrón en un string.
- Contar el número concurrencias de un patrón.
- Conocer el número de substrings diferentes en una cadena.
- Comprimir un string.
- Construir o personalizar el autómata de acuerdo a la función prefijo.

```
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = a; i < (b); ++i)
typedef long long ll;

vector<int> pi(const string& s) {
    vector<int> p(s.size());
    for (int i = 1; i < s.size(); i++) {
        int g = p[i - 1];
        while (g && s[i] != s[g])
            g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}

vector<int> match(const string& s, const string& pat) {
    vector<int> p = pi(pat + '\0' + s), res;
    for (int i = p.size() - s.size(); i < p.size(); i++) {
        if (p[i] == pat.size())
            res.push_back(i - 2 * pat.size());
    }
    return res;
}
```

9.2 Aho-Corasick

Descripción: Nos permite buscar rápidamente múltiples patrones en un texto. El conjunto de patrones se denomina diccionario. El algoritmo construye un autómata de estados finitos basado en un trie (árbol de prefijos) y luego lo utiliza para procesar el texto.

Complejidad Temporal: $O(m * k)$ donde m es la longitud total de las cadenas y k el tamaño del alfabeto.

Usos:

- Encontrar todas las cadenas de un conjunto dado en un texto.
- Encontrar la cadena lexicográficamente más pequeña de una longitud dada que no coincida con ninguna cadena dada.
- Encontrar la cadena más corta que contiene todas las cadenas dadas
- Encontrar la cadena de longitud lexicográficamente más pequeña de longitud L que contiene K strings

```
/**
 * Autor: Simon Lindholm
 * Fecha: 2015-02-18
 * Licencia: CC0
 * Código: marian's (TC) code
 * Descripción: Autómata de Aho-Corasick, utilizado para
 * búsqueda de múltiples patrones.
 * Se inicializa con AhoCorasick ac(patterns); el nodo inicial
 * del autómata estará en el índice 0.
 * find(word) devuelve, para cada posición, el índice de la palabra
 * más larga que termina ahí, o -1 si no hay ninguna.
 * findAll($-$, word) encuentra todas las palabras
 * (hasta $N \sqrt{N}$ si no hay patrones duplicados) que comienzan en
 * cada posición (las más cortas primero).
 * Se permiten patrones duplicados; no se permiten patrones vacíos.
 * Para encontrar las palabras más largas que comienzan en cada posición,
 * invierte todas las entradas.
 * Para alfabetos grandes, divide cada símbolo en partes, con
 * bits centinela para marcar los límites de los símbolos.
 * Tiempo: la construcción toma  $O(26N)$ , donde  $N = \sum$  suma de
 * las longitudes de los patrones.
 * find(x) es  $O(N)$ , donde N = longitud de x. findAll es  $O(NM)$ .
 */
```

```
#pragma once

#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = a; i < (b); ++i)
typedef long long ll;

// Estructura principal del autómata de Aho-Corasick
struct AhoCorasick {
    // Tamaño del alfabeto (por defecto 26 letras mayúsculas)
    // Caracter base es 'A' pero puede ser otro, cambiar de ser necesario
    enum { alpha = 26,
           first = 'A' };

    // Estructura de cada nodo en el trie
    struct Node {
        int back; // Enlace de fallo, a dónde debo saltar (fail link)
        int next[alpha]; // Enlaces a hijos (para cada letra del alfabeto)
        int start = -1; // Índice del primer patrón que
                        // termina en este nodo
        int end = -1; // Último patrón que termina en este nodo
        int nMatches = 0; // Cantidad de patrones que terminan aquí
                        // (incluyendo por fail links)

        // Inicializa hijos con valor 'v'
        Node(int v) { memset(next, v, sizeof(next)); }
    };

    vector<Node> N; // Lista de nodos del trie
    vector<int> backp; // Enlaces hacia patrones anteriores
                      // (para patrones duplicados)

    // Inserta un patrón en el trie
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0; // Empezar en el nodo raíz

        for (char c : s) {
            // Referencia al hijo correspondiente al carácter actual
            int& m = N[n].next[c - first];
            if (m == -1) {
                // Si no existe el nodo hijo, lo creamos
                n = m = N.size();
                N.emplace_back(-1);
            } else {
                n = m; // Continuamos al hijo existente
            }
        }

        // Si es la primera vez que se alcanza
        // este nodo como final de un patrón
        if (N[n].end == -1) N[n].start = j;
    }
};
```

```
// Mantenemos enlace a patrón anterior si hay duplicados
backp.push_back(N[n].end);
N[n].end = j; // Marcamos que este patrón termina aquí
N[n].nMatches++; // Aumentamos la cantidad de coincidencias
}

// Constructor: recibe un conjunto de patrones y construye el autómata
AhoCorasick(vector<string>& pat) : N(1, -1) {
    // Insertar todos los patrones en el trie
    rep(i, 0, pat.size()) insert(pat[i], i);

    // Inicializar nodo raíz con enlace a un nodo dummy adicional
    N[0].back = N.size();
    N.emplace_back(0);

    // Construcción de los enlaces de fallo (fail links)
    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
        int n = q.front(), prev = N[n].back;
        rep(i, 0, alpha) {
            int& ed = N[n].next[i]; // Hijo actual
            int y = N[prev].next[i]; // Hijo del nodo de fallo

            if (ed == -1) {
                // Si no hay hijo, redirigir a hijo del nodo de fallo
                ed = y;
            } else {
                // Si sí hay hijo, se construye su fail link
                // y se actualizan coincidencias
                N[ed].back = y;

                // Actualizar enlaces de patrones duplicados (si los hay)
                (N[ed].end == -1 ? N[ed].end : backp[N[ed].start]) =
                    N[y].end;

                // Acumular cantidad de coincidencias desde el fail link
                N[ed].nMatches += N[y].nMatches;

                // Añadir hijo a la cola para procesar sus hijos
                q.push(ed);
            }
        }
    }

    // Búsqueda de patrones en un texto. Devuelve un vector con el
    // índice del patrón más largo que termina en cada posición
    // del texto, o -1 si no hay ninguno.
    vector<int> find(string word) {
        int n = 0; // Comenzar desde la raíz
        vector<int> res;
        // ll count = 0;
    }
}
```

```

    for (char c : word) {
        // Avanzar al siguiente nodo
        n = N[n].next[c - first];
        // Guardar el patrón (si hay alguno que termina aquí)
        res.push_back(N[n].end);

        // count += N[n].nMatches;
    }
    return res;
}

// Búsqueda de **todos** los patrones que
// terminan en cada posición del texto.
vector<vector<int>> findAll(vector<string>& pat, string word) {
    vector<int> r = find(word);
    vector<vector<int>> res(word.size());

    rep(i, 0, word.size()) {
        int ind = r[i];
        // Seguir los enlaces hacia atrás (backp) para
        // encontrar patrones anteriores
        while (ind != -1) {
            // Marcar la posición donde inicia el patrón
            res[i - pat[ind].size() + 1].push_back(ind);
            // Saltar a un patrón duplicado anterior (si lo hay)
            ind = backp[ind];
        }
    }
    return res;
}
};

int main() {
    // Lista de patrones a buscar
    vector<string> patrones = {"HOLA", "MUNDO", "LO", "NO"};

    // Crear el autómata
    AhoCorasick ac(patrones);

    // Texto donde se van a buscar los patrones
    string texto = "HOLAMUNDONOMUNDO";

    // Buscar todas las ocurrencias de los patrones en el texto
    vector<vector<int>> resultados = ac.findAll(patrones, texto);

    // Mostrar resultados
    for (int i = 0; i < resultados.size(); ++i) {
        for (int id : resultados[i]) {
            cout << "Patron_" << patrones[id]
                 << "_encontrado_en_posicion_" << i << '\n';
        }
    }
}

```

```

/** Impresión
 * Patron HOLA encontrado en posicion 0
 * Patron MUNDO encontrado en posicion 4
 * Patron NO encontrado en posicion 9
 * Patron MUNDO encontrado en posicion 11
 * */

return 0;
}

```

10 Lorem ipsum

10.1 Código de ejemplo

Descripción: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Complejidad Temporal: $O(n\log(n) + m^2)$

Usos: Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

```
#include <iostream>

int main() {
    std::cout << "Hola_Mundo" << std::endl;
    return 0;
}
```